



**BOOK MY SHOW SYSTEM
(ONLINE MOVIE TICKET BOOKING SITE)
USING JAVA FULL STACK DEVELOPMENT
(JAVA SPRING BOOT)**



**PROJECT REPORT SUBMITTED IN PARTIAL
FULFILLMENT OF THE INTERNSHIP FOR THE DEGREE
OF B.TECH IN CSE(ARTIFICIAL INTELLIGENCE AND
MACHINE LEARNING) OF
MAKAUT
BY
SAYAN DAS**

**FUTURE INSTITUTE OF TECHNOLOGY
240 BORAL MAIN ROAD, KOLKATA – 700154
ACADEMIC YEAR – 2021-2025**

Book My Show System

(Online Movie Ticket Booking Site)

Using Full Stack Development with Java(Spring Boot)

**Project Report submitted in partial fulfillment of the
requirements for the degree of
Bachelor of Technology in Computer Science & Engineering (Artificial Intelligence
& Machine Learning)**

of

**Maulana Abul Kalam Azad University of Technology Under the
guidance of**

Anindya Mukherjee

Project Carried Out At



Ardent Computech Pvt Ltd(AnISO9001:2008Certified)
Module-132, SDF Building, Sector-V, Salt Lake City, Kolkata-700091

By

SAYAN DAS (University Roll-34230821008)



**Future Institute of Technology Garia,
Kolkata – 700154
Academic Year of Pass out:2021-2025**

Book My Show System

(Online Movie Ticket Booking Site)

Project Submitted by:

Sayan Das (University Roll-34230821008)

Under the guidance of:

Prof. Anindya Mukherjee

DEPARTMENT OF

BTech CSE(AIML)

**FUTURE INSTITUTE OF TECHNOLOGY
240 BORAL, GARIA, KOLKATA-700154**

This Project Thesis is submitted in partial fulfillment of the requirement for the Bachelor of Technology in Computer Science And Engineering (Artificial Intelligence And Machine Learning) under West Bengal University of Technology.

Project Submitted By: Sayan Das

***Signature*_____**

Under the guidance of: Mr. Anindya Mukherjee

***signature of the Project guide*_____**

Department of BTech CSE(AIML)
Future Institute of Technology
240 Boral, Garia, Kolkata – 700154.

CERTIFICATE

This is to certify that this project report titled **Book My Show System (Online Movie Ticket Booking) Using Java Full Stack Development (Java Spring Boot)** Submitted in partial fulfillment of requirements for award of the degree Bachelor of Technology in Computer Science And Engineering (Artificial Intelligence And Machine Learning) of Maulana Abul Kalam Azad University of Technology is a faithful record of the original work carried out by –

SAYAN DAS
REGISTRATION NO: 213420130810012 of 2021-22
UNIVERSITY ROLL NUMBER - 34230821008

Under my guidance and supervision. It is further certified that it contains no material, which to a substantial extent has been submitted for the award of any degree in any institute or has been published in any form, except the assistances drawn from other sources, for which due acknowledgement has been made.

(Signature of HOD)

(Signature of project guide)

.....

.....

Dr. Pradipta Kr. Banerjee

Mr. Anindya Mukherjee

FIT
240 Boral, Garia, Kolkata-700154

DECLARATION

We hereby declare that this project report titled
**Book My Show System (Online Movie Ticket Booking Site) Using Java
Full Stack Development (Java Spring Boot)**

Is our own original work carried out as a under graduate student in Future
Institute of Technology except to the extent that assistances from other sources
are duly acknowledged.

All sources used for this project report have been fully and properly cited. It
contains no material which to a substantial extent has been submitted for the
award of any degree in any institute or has been published in any form, except
where due acknowledgement is made.

Students Name

Signature

Dates

.....

.....

CERTIFICATE OF APPROVAL

We hereby approve this dissertation titled **Book My Show System
(Online Movie Ticket Site) Using Java Full Stack Development (Java
Spring Boot)**

Carried out by

Sayan Das
Reg.No-213420130810012 of 2021-22
Roll No-34230821008

Under the guidance of

Mr. Anindya Mukherjee

Of Ardent Private Limited,
Kolkata in partial fulfillment of requirements for award of the Bachelor of
Technology in Computer Science and Engineering (Artificial Intelligence And
Machine Learning) of Maulana Abul Kalam Azad University of Technology West
Bengal.

Date:

Examiner's Signatures:

1.

2.

ACKNOWLEDGEMENTS

The achievement that is associated with the successful Completion of any task would be incomplete without mentioning the name of **Sayan Das** whose endless cooperation made it possible. His constant guidance and encouragement made all our efforts successful. We take this opportunity to express our deep gratitude towards our project mentor **Mr. Anindya Mukherjee** for giving such valuable suggestions guidance and encouragement during the development of this project work. Last but not the least we are grateful to all the faculty members of **Engineering Study Centre** for their support.

Dated:

Sayan Das

INTRODUCTION

The Book My Show System can be defined as a simple and convenient way for customers to purchase Movie Tickets online, instead of going to the theaters.

This system is enabled by the internet – it is the internet that connects the theatre company on one hand, and the customer on other hand.

Therefore, as per this system, the customer visits the booking website, browses through the various movie tickets, seats and prices available there and goes ahead and selects and purchases the tickets he or she needs.

These tickets will then be confirmed to the customer after he or she pays for the tickets selected at time of buying.

Payments for such online booking can be made through debit cards, credit cards, UPI or even through digital wallets.

This system for online Booking System is completely safe, secure and it is a very popular method that is revolutionizing the way in which the Theatre Industry operates.

ABSTRACT

In the digital age, online ticket booking platforms have revolutionized the way audiences engage with entertainment events. Book My Show stands at the forefront, offering a comprehensive online platform for discovering and booking tickets for a myriad of events. This abstract outlines a Java Spring Boot project aimed at enhancing the functionality and performance of Book My Show's online platform.

The project leverages Java Spring Boot, a robust and widely used framework for building enterprise-grade applications, to develop and deploy new features and improvements to the Book My Show website. Key objectives of the project include optimizing the booking process, improving search functionality, and enhancing overall user experience.

Visitors to the site, once registered, are then able to easily navigate this movie tickets, and buy tickets options with only a few clicks, greatly simplifying the buying process. The purpose of this document is to provide in-depth descriptions of design and implementation details of the system, as well as descriptions of all available functionality and plans for evolution.

HARDWARE AND SOFTWARE REQUIREMENTS

❖ HARDWARE :-

- HP LAPTOP 15S FQ5010TU
- PROCESSOR –INTEL I5
- RAM- 4GM
- HARDDISK-1000 GB

❖ OPERATING SYSTEM USED :-

- WINDOWS 11 64 BIT

❖ TOOLS USED :-

- SPRING TOOL SUITE 4
- NOTEPAD

❖ FRAMEWORK USED :-

- JAVA SPRING BOOT

❖ LANGUAGES USED:-

- HTML
- CSS
- MYSQL
- BOOTSTRAP
- JAVASCRIPT
- JAVA
- JSP SERVLET

❖ DATABASE USED :-

- MYSQL

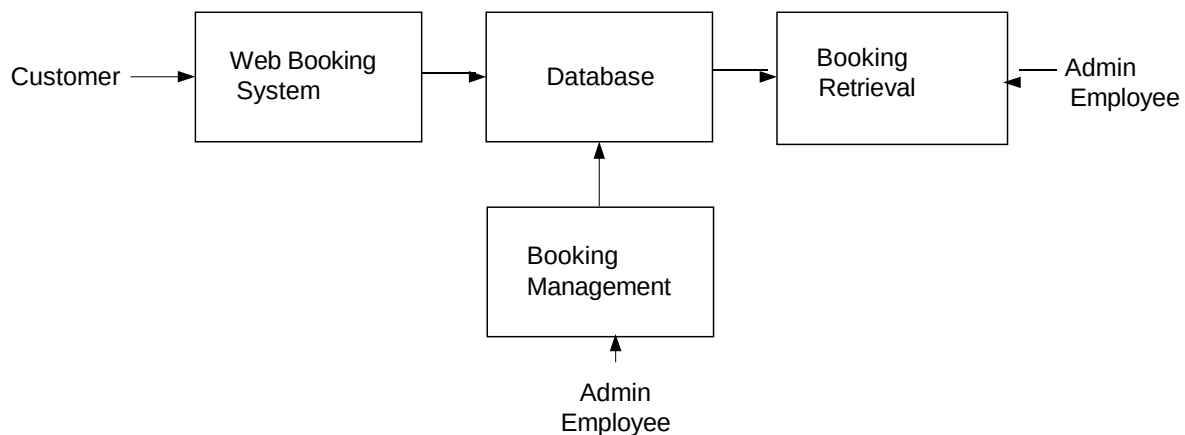
❖ JAVA VERSION:-

- JAVA VERSION 17

Requirements Specification :-

System Model -

The structure of the system can be divided into three main logical components. The first component must provide some form of booking management, allowing the theatre to control which movies can be booked by the customers. The second component is the web booking system and provides the functionality for customers to place their tickets and supply all necessary details. The third and final logical component is the booking retrieval system. Used by the theatre to keep track of all booking which have been booked, this component takes care of retrieving and displaying booking information, as well as updating booking which have already been processed.



Functional Requirements -

As can be seen in the system model diagrammed above, each of the three system components essentially provides a layer of isolation between the end user and the database. The motivation behind this isolation is two fold. Firstly, allowing the end user to interact with the system through a rich interface provided a much more enjoyable user experience, particularly for the non-technical users which will account for the majority of the system's users. In addition, this isolation layer also protects the integrity of the database by preventing users from taking any action outside those which the system is designed to handle. Because of this design pattern, it is essential to enumerate exactly which functions a user will be presented and these functions are outlined below, grouped by component.

The Booking Ordering System:-

Users of the booking ordering system, for theatre's customers, must be provided the following functionality:

- Create an account.
- Manage their account.
- Log in to the system.
- Navigate the movie list.
- Select a movie from the list.
- Select seats according to their preferences and the availability
- Book tickets for their current selected movie.
- Provide payment details.
- Book Tickets.
- Receive confirmation.

As the goal of the system is to make the process of booking tickets is as simple as possible for the customer, the functionality provided through the web booking system is restricted to that which most pertinent to accomplish the desired task. All of the functions outlined above, with the exceptions of account creation and management, will be used every time a customer books a ticket. By not including extraneous functions, I am moving towards my goal of simplifying the booking process .

Booking Management System: -

The booking management system will be available only to admin employees as the name suggests, it allows them to manage the movie list that is displayed to users of the web booking system. The functions afforded by the booking management system provide user with the ability to, using a graphical interface:

- Add a new/update/delete movie to/from the movie category list.
- Update price for a given movie tickets.
- Update/delete number of available seats for a movie.
- Update additional information (description, photo, etc.) for a given movie.

It is anticipated that the functionality provided by this component will be one of the first things noted by the admin user, as they will have to go through it to configure their movie list, etc. before beginning to actually booking the ticket. Once everything is initially configured, however, this component will likely to be the least used, as movie list updates generally do not occur with great frequency.

Booking Retrieval System :-

Of the three components, the booking retrieval system is functionally the simplest. Like the booking management system, it is designed to be used only by admin employees, and provides the following functions:

- Retrieve new booking from the database.
- Display the booking in an easily readable, graphical way.
- Mark a booking as having been processed and remove it from the list of active booking.

User Interface Specifications -

Each of the system components will have their own unique interface. These are described below.

Web Booking System-

Users of the web booking system will interact with the application through a series of simple forms. Each category of movie has its own form associated with it which presents a drop down for choosing which specific movie from the category and book. Booking a ticket to buy is accomplished by a single button click. Entering payment deals is done in a similar manner. The user is presented with a form and must complete the required fields.

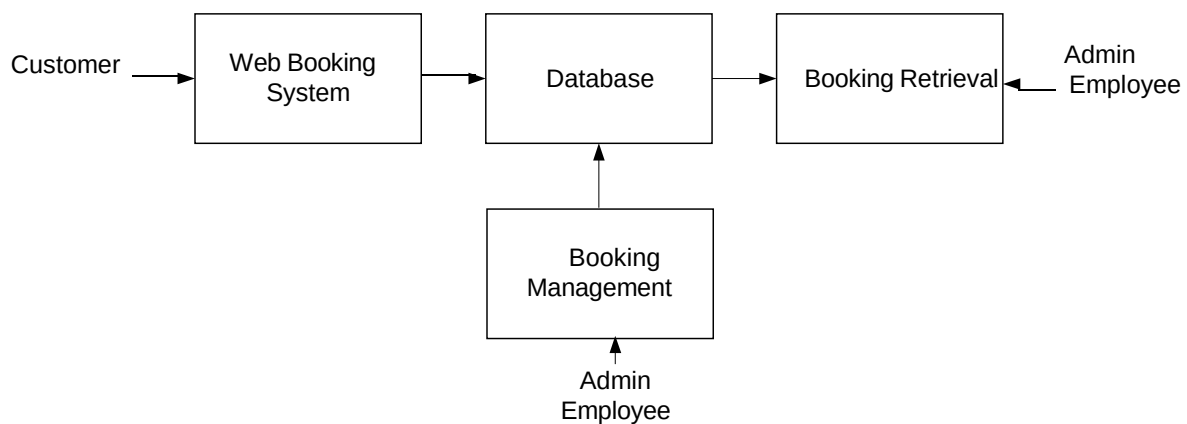
Booking Management System-

User interaction with the booking management system is similar to that with the web booking system. Users navigate a tree structure to find customer , category, or specific booking that they would like to modify and after making their selection they are presented with a form which displays all of the current fields and values associated with that movie, all of which can be modified or removed. The form also presents buttons which allow the addition of new fields and values. Unlike the web booking system, however, most of the input here will be freeform specifically in the form of text boxes, since there is no finite set of fields which could be added.

System Design :-

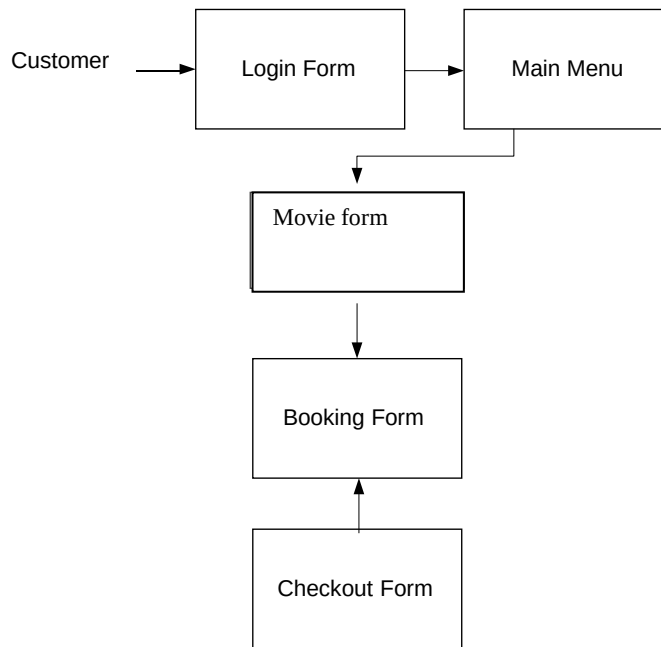
Level 1: The Database & the 3 Components

The structure of the system can be divided into three main logical components. The first component must provide some form of booking management, allowing the theatre to control which movies can be booked by the customers. The second component is the web booking system and provides the functionality for customers to place their tickets and supply all necessary details. The third and final logical component is the booking retrieval system. Used by the theatre to keep track of all booking which have been booked, this component takes care of retrieving and displaying booking information, as well as updating booking which have already been processed.



Level 2: Web Ordering System Components

The web ordering system is comprised of 5 major components. These are the login form, the main menu, the movie form, the booking form, and the checkout form. When the customer first arrives at the site, they are presented with the login form. After either signing in or, if they do not yet have an account, first registering and then signing in, the user is taken to a welcome page with the main menu. From here, they can navigate the numerous movie posters using the main menu, each of which corresponds to a specific category of movie. After this they can proceed with selecting the seats according to their preferences, availability, show time and theatre of their choice. Then they can proceed to the checkout form. The checkout form uses the contents of the selected seats to present a summary of the booking and to calculate the total cost, in addition to allowing the user to specify all of the necessary details.



Level 2: The Login Form

The login form is standard for a form of this type. It provides text fields for username and password, which the user must enter before signing in. This form also gives the option for a user to register for the site if they have not yet done so.

Level 2: The Main Menu

The main menu, found at the top of the screen like in most applications, presents the user with two levels of selections. They must first choose the movie according to the and available shows and the category he/she wishes, then the website generates a movie form for the specific movie and displays this form to the user.

Level 2: The Movie Form

The movie form, which is dynamically generated based on selections from the main menu,

Level 2: The Booking Form

The booking form performs like a normal booking website. In the booking form, the customer selects the seats according to their showtime, preferences and availability.

After selecting the seats, the booking form contains a option for the user to proceed to checkout.

Level 2: The Checkout Form

The checkout form is the user's last chance to verify that the contents of their booking are correct or not. This form also provides fields for the user to supply all of the necessary checkout details (payment mode)

Database-

Testing of the database component is very straightforward, and has actually already been mostly completed.

The database was the first component designed and before beginning work on any of the applications, I wrote all of the SQL statements I expected to need and executed them directly, essentially isolating the database, using the MySQL client.

By doing this I was able to reveal, and promptly fix a large percentage of the errors within the database itself.

DFD (Data Flow Diagram) :-

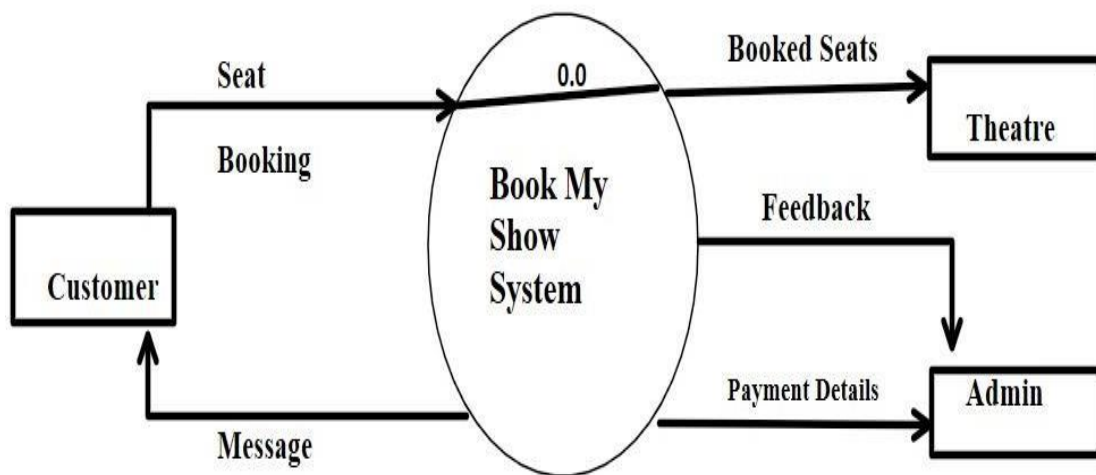
Book My Show System is actually a type of website that allows the admin of Book My Show to manage newly added movies, show times in the theatre of the newly as well as the previous released movies and accept the seat bookings over the Internet. Let us understand the working of the Book My Show system by using DFD.

Here, different levels of DFD are shown for Book My Show System such as Level 0 DFD, Level 1 DFD.

Level 0 DFD –

At this level, the Input and Output of the system are shown. The system is designed and established across the world with input and output at this level.

Level 0 DFD



Book My Show System has the following input :

Seat Booking is the input as the customer's booking for movie.

Book My Show System has the following output:

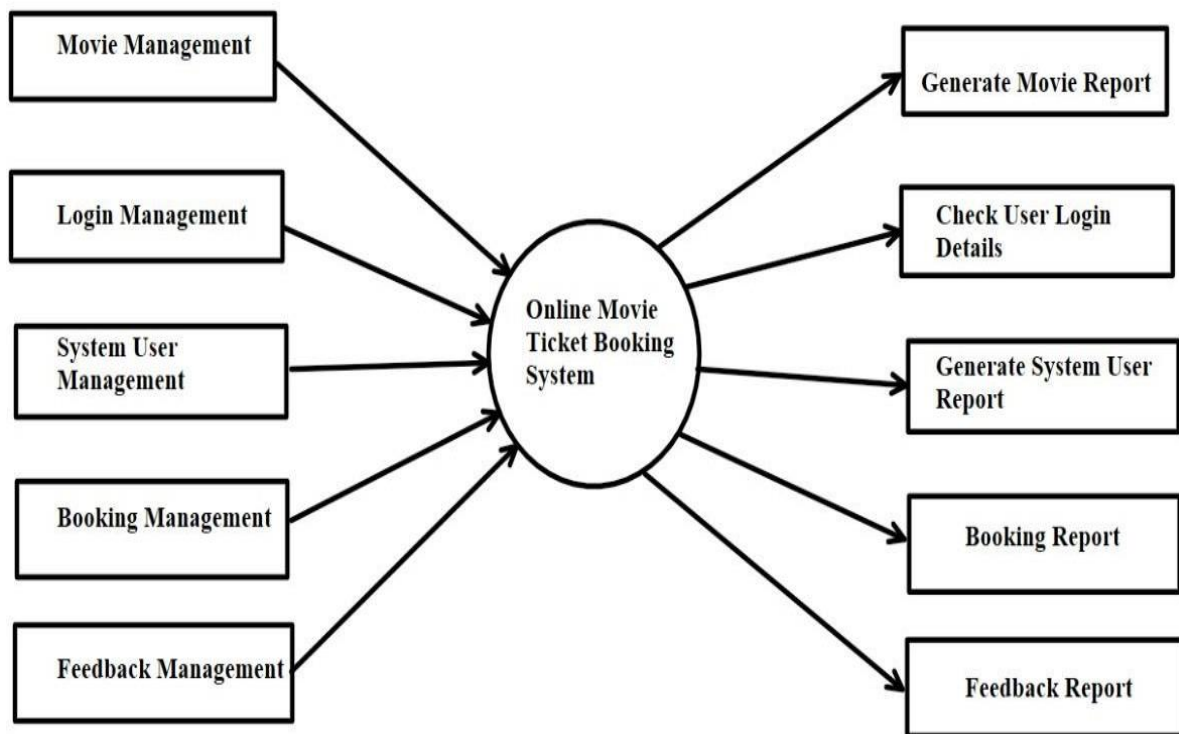
- Message of the Booked Seats.
- For further processing, the booked seat is passed to the theatre.
- The admin gets the report of payment details and the feedback by the customer.

Level 1 DFD –

First Level DFD (1st Level) of Online Movie Ticket Booking System shows how the system is divided into sub-systems (processes), each of which deals with one or more of the data flows to or from an external agent, and which together provide all of the functionality of the Online Movie Ticket Booking System as a whole.

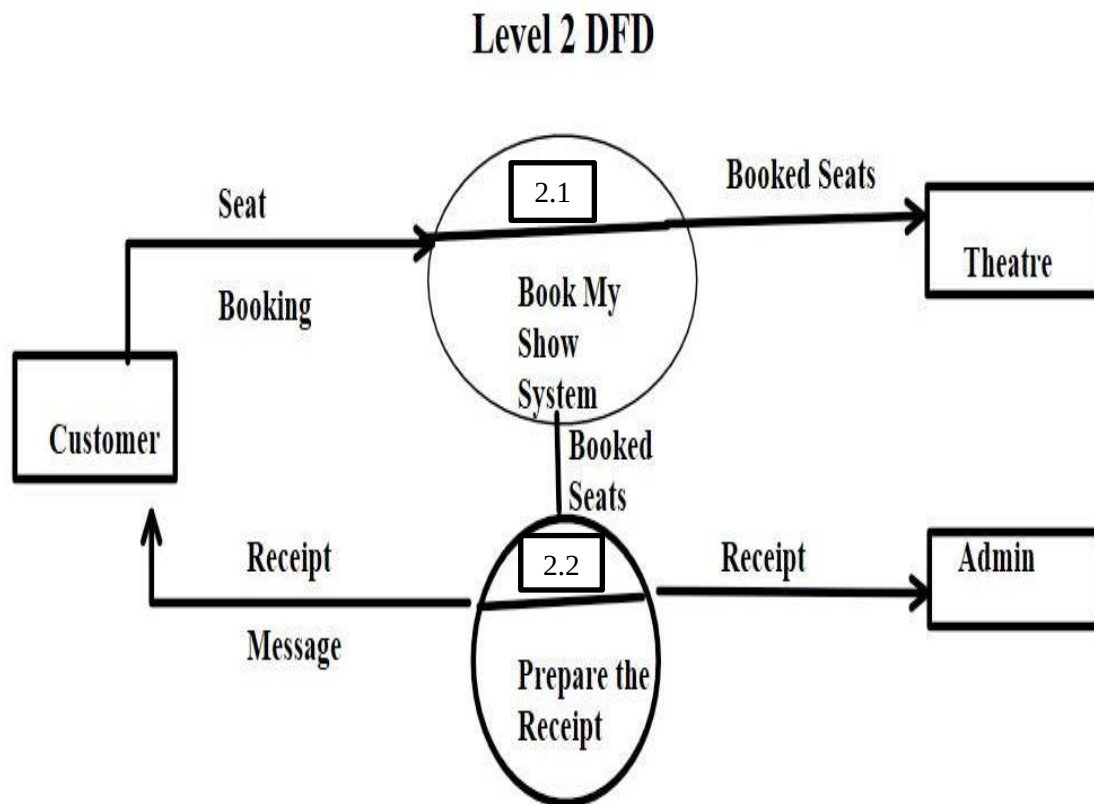
It also identifies internal data stores of Seats, Booking that must be present in order for the Movie Ticket system to do its job, and shows the flow of data between the various parts of Movie, Booking, Seats of the system. DFD Level 1 provides a more detailed breakout of pieces of the 1st level DFD. You will highlight the main functionalities of Movie Ticket

Level 1 DFD



Level 2 DFD –

Detailed information about “Processing of an Booking” is shown below :



ER Diagram For Book My Show System :-

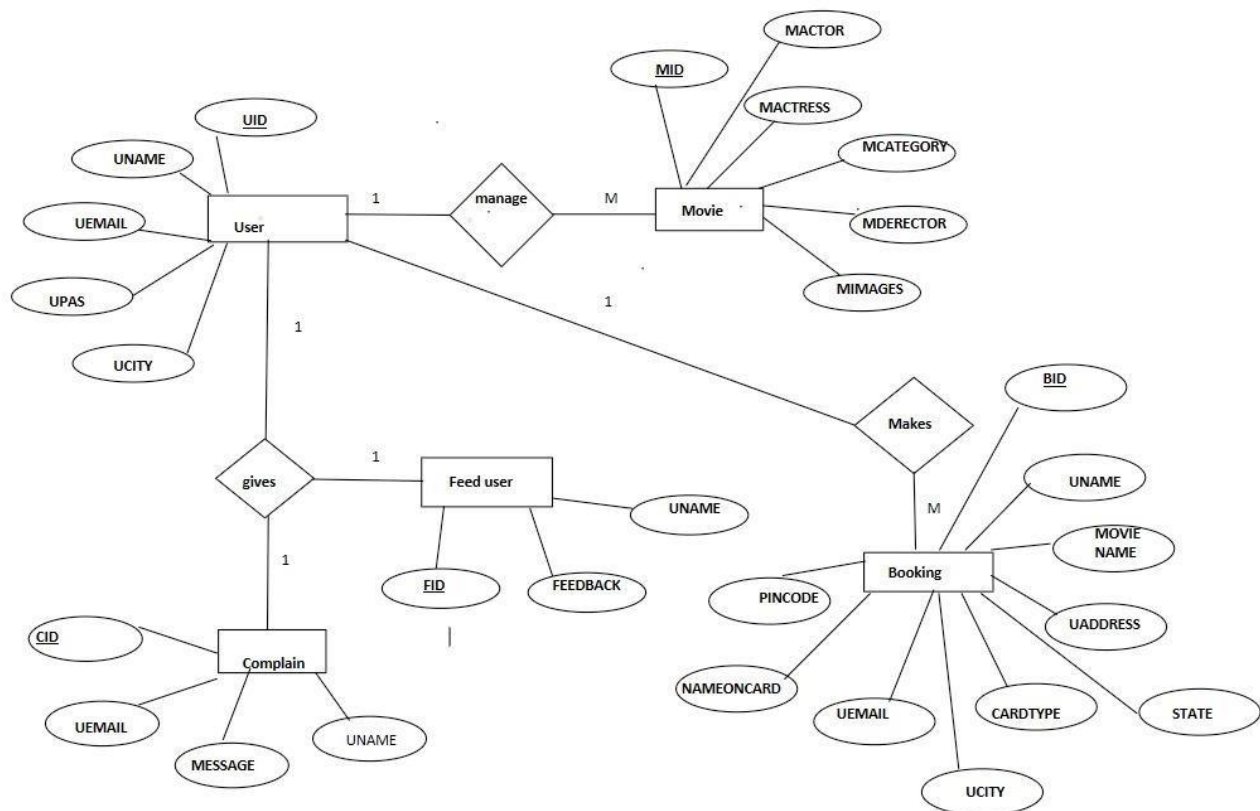
Entity is represented by the rectangle shape. The **entity** will be our **database** table of **Online Booking System** later on. Attribute is represented by the oval shape. This will be the columns or fields of each table in the **Online Booking System**.

We will follow the 3 basic rules in creating the ER Diagram.

1. Identify all the entities.
2. Identify the relationship between entities and
3. Add meaningful attributes to our entities.

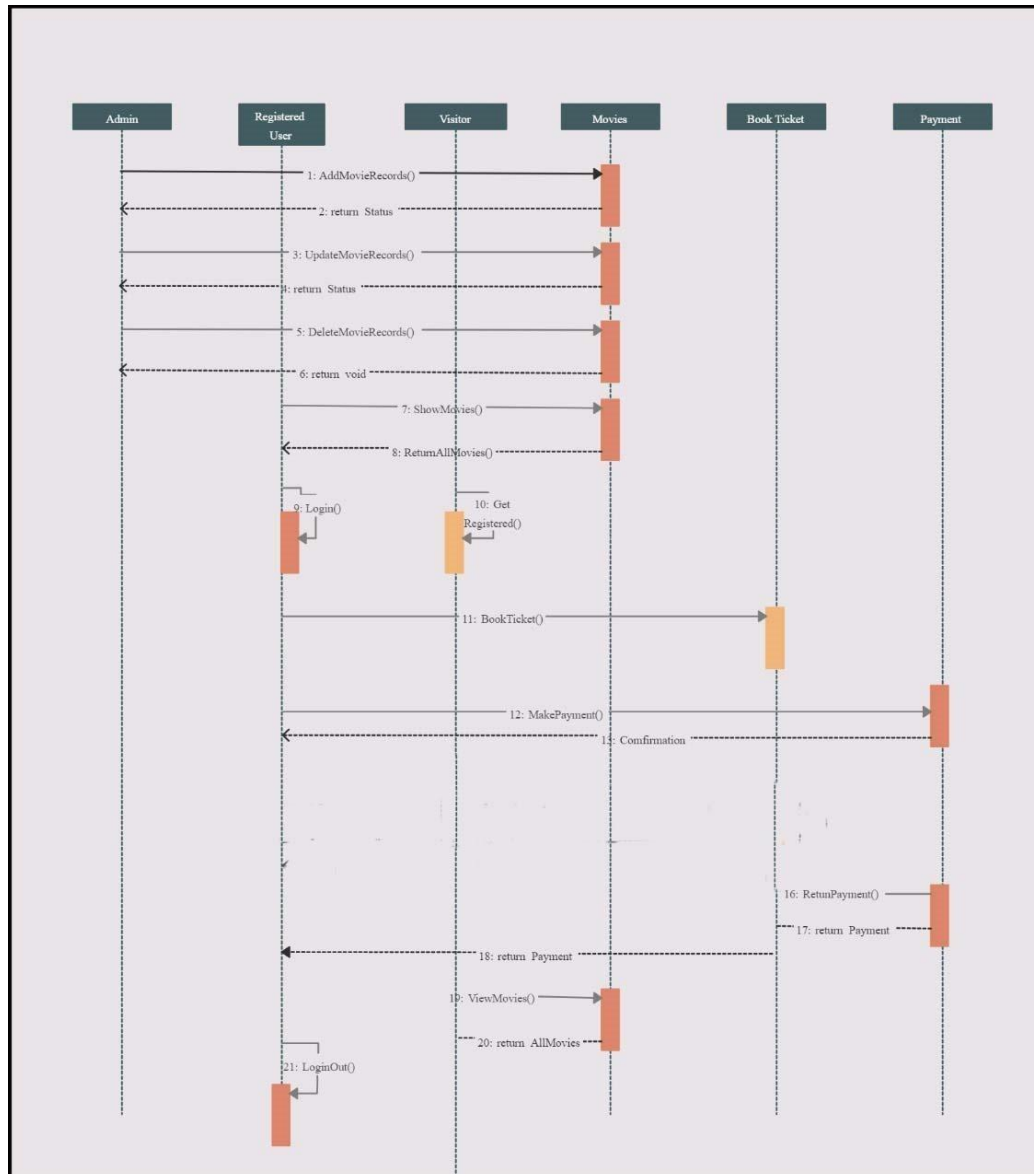
Step 1. In the Online Movie Booking System we have the following entities

- User
- Movie
- Payment
- Complain



Sequence Diagram For Book My Show System :-

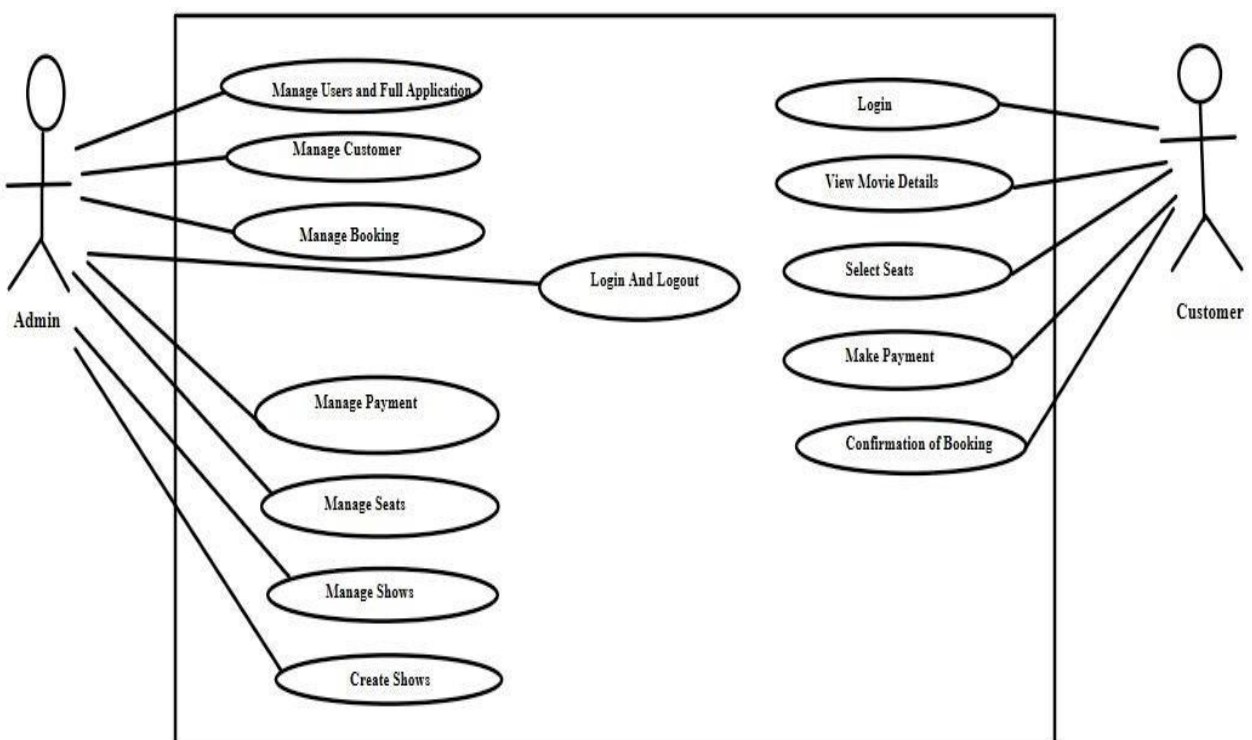
This is the **UML sequence diagram** of **Book My Show System** which shows the interaction between the objects of Admin, User, Visitor, Movies, Payment. The instance of class objects involved in this **UML Sequence Diagram** of **Book My Show System** are as follows:



Use-Case Diagram For Book My Show System :-

This **Use Case Diagram** is a graphic depiction of the interactions among the elements of **Book My Show System**. It represents the methodology **used** in **system** analysis to identify, clarify, and organize **system** requirements of **Book My Show System**. The main actors of Book My Show System in this Use Case Diagram are: Admin, Customer, who perform the different type of use cases such as Manage Customer and Full Application, Manage Booking, Manage Payment, Manage Seat, Manage Shows, Create Shows. Major elements of the UML use case diagram of Book My Show System are shown on the picture below.

Use Case Diagram of Book My Show System :



Online movie booking systems offer several advantages and disadvantages:

Advantages:

- **Convenience:** One of the most significant advantages of online movie booking systems is the convenience they offer. Customers can book tickets anytime, anywhere, without having to visit the cinema physically.
- **Time-saving:** Online booking eliminates the need to stand in long queues at the ticket counter, saving customers valuable time.
- **Seat selection:** Many online booking systems allow customers to choose their preferred seats from an interactive seating map, ensuring they get their desired spot in the cinema hall.
- **Easy payment options:** Customers can pay for their tickets online using various payment methods, including credit/debit cards, digital wallets, and net banking, making the transaction process hassle-free.
- **Accessibility:** Online booking systems are accessible to a wide range of users, including those with disabilities, as they can be customized to accommodate various needs.
- **Movie information:** Customers can easily access information about upcoming movies, show timings, trailers, and reviews through online booking platforms, helping them make informed decisions.

Disadvantages:

- **Internet dependency:** Online booking systems require a stable internet connection, which may not be available in all areas or to all users, limiting access to the service.
- **Technical issues:** Glitches, server crashes, or other technical problems can occasionally disrupt the booking process, leading to frustration for customers and potential loss of business for the cinema.
- **Service fees:** Some online booking platforms charge convenience fees or service charges on top of the ticket price, which may discourage cost-conscious customers from using the service.
- **Privacy concerns:** Customers may have reservations about providing personal and financial information online, especially if they perceive the platform to be insecure.
- **Limited options:** While online booking systems offer convenience, they may not always provide the same level of flexibility as purchasing tickets directly from the cinema, particularly regarding last-minute changes or cancellations.
- **Exclusivity:** Some cinemas may offer exclusive deals or discounts only for tickets purchased at the box office, which could discourage customers from using online booking platforms.

FUTURE SCOPES OF BOOK MY SHOW SYSTEM:

- Cancellation of tickets.
- Updating and Deleting Movies by the Admin.
- Booking confirmation will be send through registered Email.
- Adding food menu choice option with booking tickets.
- Customizing more seat booking facilities.
- Show hall details with time for each movie.
- Adding the facilities of logout for customers.
- Booking of other events such as Sports Match, Musical Concerts will also be provided.
- Change Password Facility will be provided for the user.

Conclusion:

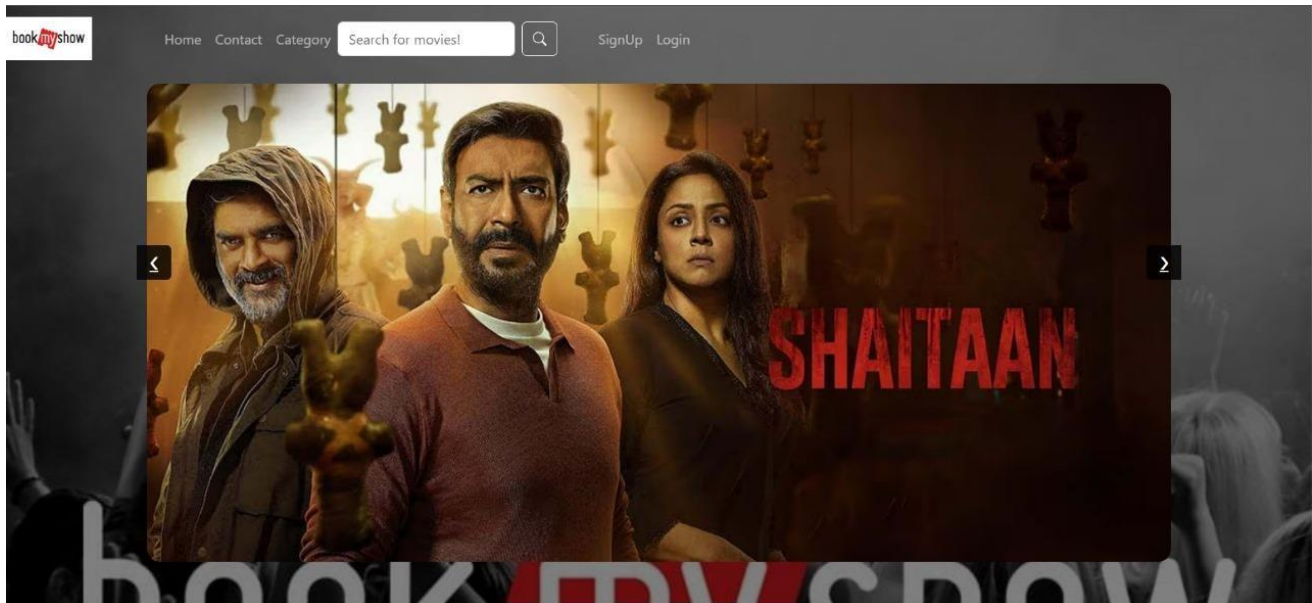
The online movie ticket booking system has revolutionized how audiences engage with cinema, offering unparalleled convenience and accessibility. It enables users to effortlessly book tickets anytime, anywhere, saving time and eliminating the hassle of queues. However, challenges such as internet dependency, technical glitches, and privacy concerns persist. Despite this, the system's advantages including seat selection, easy payments, and access to movie information outweigh its drawbacks. Continuous advancements in technology promise to enhance the user experience further. Overall, the online movie ticket booking system stands as a cornerstone of modern entertainment, reshaping how audiences experience and interact with the cinematic world.

Bibliography:

- <https://www.freeprojectz.com>
- <https://www.w3schools.com/>
- <https://github.com/>
- <https://stackoverflow.com/>
- <https://www.educative.io/>
- <https://www.codingpoweryt.com/>

Screen Shots Of Project Views :-

HOME PAGE



SIGNUP PAGE

The screenshot shows the 'Sign up' page of the 'bookmyshow' website. The header is dark grey with the 'bookmyshow' logo on the left. Navigation links include 'Home', 'All Movies', 'Contact', a search bar with the placeholder 'Search for movies!', and 'SignUp' and 'Login' links. The main content area has a red background with the title 'Sign up' in large, bold, white letters. Below the title, there are six input fields for user registration: 'Your Name', 'Your Email', 'Your Mobile Number', 'Password', 'Confirm Your Password', and 'City'. Each field is preceded by a small icon representing the field type (person, envelope, phone, lock, key, and building). To the right of the input fields, there is a large, rounded rectangular graphic containing a movie reel, a bucket of popcorn, a yellow movie ticket with the word 'CINEMA' on it, and a pair of 3D glasses.

USER LOGIN SECTION



User Login Page

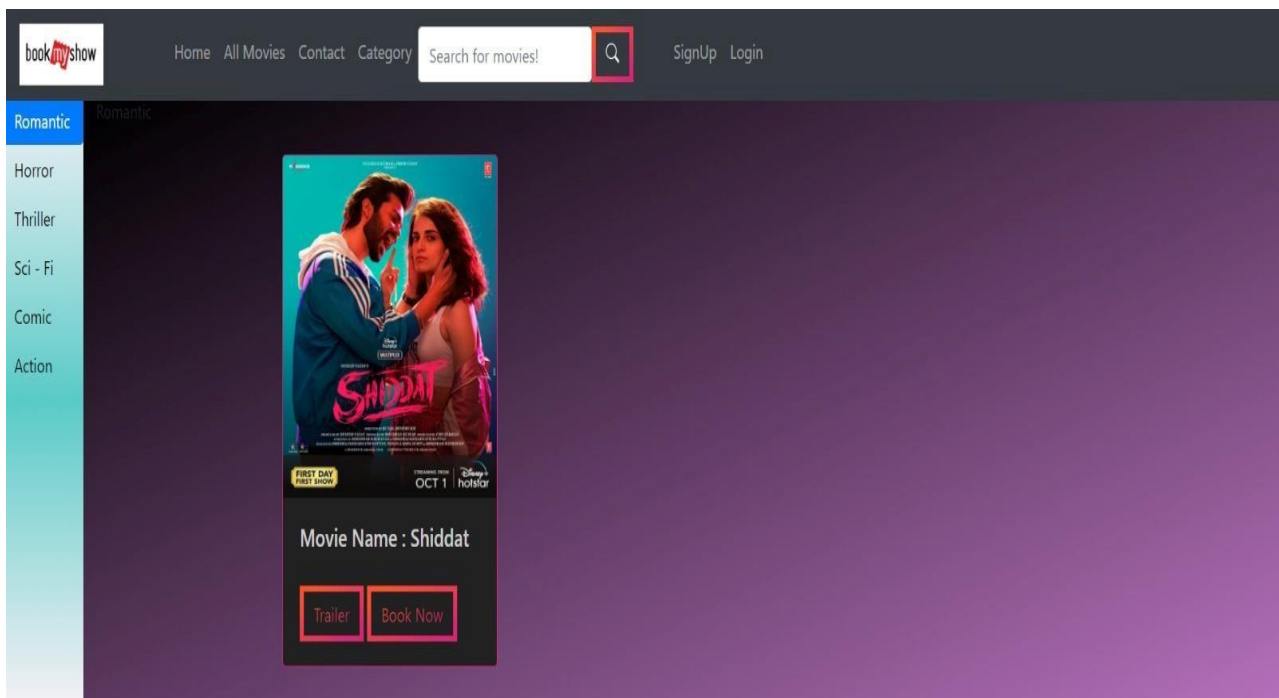
Don't have an account [Sign-in](#)

Email:

Password:

[login](#)

CATEGORY



SEAT SELECTION

body onload="onLoaderFunc()">

Pick a movie: Shiddat (Rs. 200) ▾

	1	2	3	4	5	6	7	8	9	10	11	12
A												
B												
C												
D		✓	✓	✓	✓	✓						
E												
F												
G												
H												
I												
J												

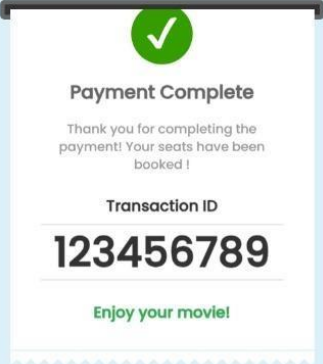
Confirm Selection

PAYMENT SECTION

Gmail YouTube Maps

Billing Address	Payment
Movie Name: Shiddat	Card Accepted:
Full Name: Prateek Kumar Singh	Name On Card: Prateek Kumar Singh
Email: Prateeksingh78264@Gmail.Com	Card Type: Debit
Address: Kidderpore	Credit Card Number: 1254-1257-7852-1369
City: Kolkata	Exp Month: October
State: West Bengal	Pin Code: 700023
	Exp Year: 2028
	CVV 1257

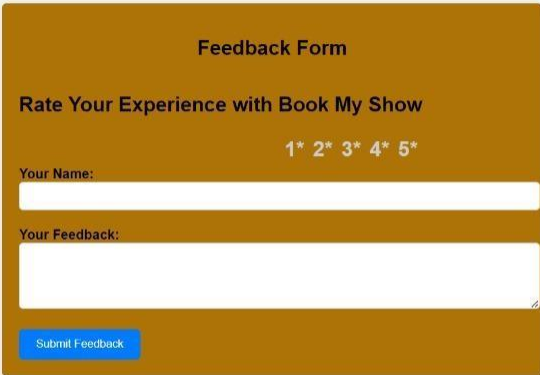
AFTER PAYMENT SUCCESS SECTION



A confirmation card with a green checkmark icon at the top. The text on the card reads: "Payment Complete", "Thank you for completing the payment! Your seats have been booked!", "Transaction ID", "123456789", and "Enjoy your movie!". The card is set against a light blue background.

[Give Feedback](#)

FEEDBACK FORM



A feedback form titled "Feedback Form" with the subtitle "Rate Your Experience with Book My Show". It includes a rating scale from 1* to 5*, a text input field for "Your Name:", a text area for "Your Feedback:", and a "Submit Feedback" button. The form is set against a light gray background.

Feedback Form

Rate Your Experience with Book My Show

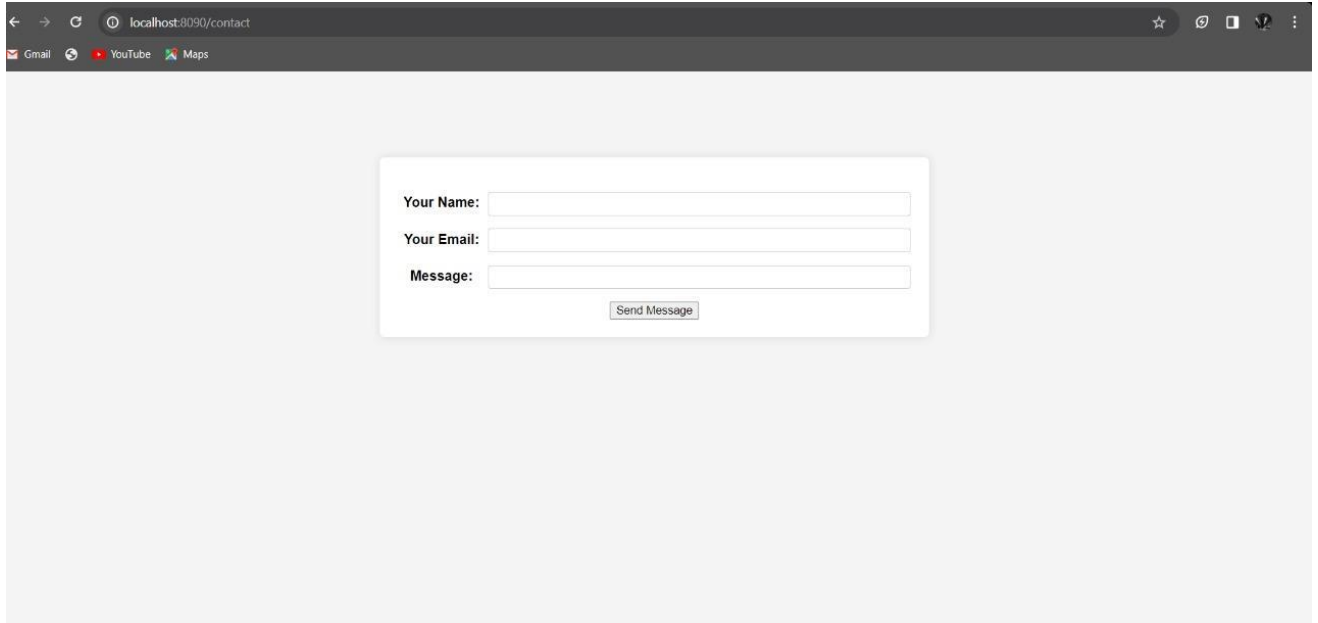
1* 2* 3* 4* 5*

Your Name:

Your Feedback:

Submit Feedback

CONTACT PAGE



A screenshot of a web browser window displaying a contact form. The browser's address bar shows the URL "localhost:8090/contact". The page has a light gray background. In the center, there is a white rectangular form with a subtle shadow. The form contains three input fields: "Your Name:", "Your Email:", and "Message:". Below these fields is a "Send Message" button. The browser's tab bar shows icons for Gmail, YouTube, and Maps.

localhost:8090/contact

Your Name:

Your Email:

Message:

Send Message

ADMIN LOGIN VIEW



A screenshot of an admin login page. The background is solid black. In the center, there is a gray rectangular box with rounded corners and a thin pink border. Inside this box, at the top, is a red rectangular button with the text "Admin Login Page" in white. Below this button are two input fields: "Email:" and "Password:". Below the "Password:" field is a blue rectangular button with the text "login" in white.

Admin Login Page

Email:

Password:

login

ADMIN ADDING MOVIES

← → ↻ 📍 localhost:8090/addmovie

Movie Id

Movie Name

Category

Romantic

▼

Ticket Quantity

Price

Actors

Actress

Director

Select File:

Choose File

No file chosen

Submit

ADMIN VIEWING ADDED MOVIES

Movie List

Movied	Catagory	MovieName	Image	Quantity	Price	Actor	Actress	Director		
101	romantic	Shiddat	Shiddat_poster.jpg	100	150.0	Sunny Kaushal, Mohit Raina	Radhika Madan, Diana Penty	Kunal Deshmukh	Update	Delete
102	Action	Fighter	fighter.jpg	500	250.0	Hrithik Roshan, Anil Kapoor	Deepika Padukone	siddharth Anand	Update	Delete
103	scifi	Gravity	gravity.jpg	50	100.0	George Clooney	Sandra Bullock	Alfonso Cuaron	Update	Delete

ADMIN DASHBOARD

bookmyshow

Book My Show

Admin Dashboard

Log-out



Add Movies



View Movies



View User



View Booking



View Feedback



View Contacts

ADMIN VIEWING REGISTERED USER

User List

Name	Email	Password	PhoneNo	City
Prateek Kumar Singh	prateek@gmail.com	admin	98745632105	Kolkata
Akash Saha	akash@gmail.com	akash	98745632105	Jaigaon

ADMIN VIEWING FEEDBACK FROM THE USER

Feedback List

Name	Feedback
Prateek Kumar Singh	good service

ADMIN VIEWING CONTACT FROM THE USER

Contact List

Name	Email	Message
Prateek Kumar Singh	prateek@gmail.com	poor server

ADMIN LOGOUT

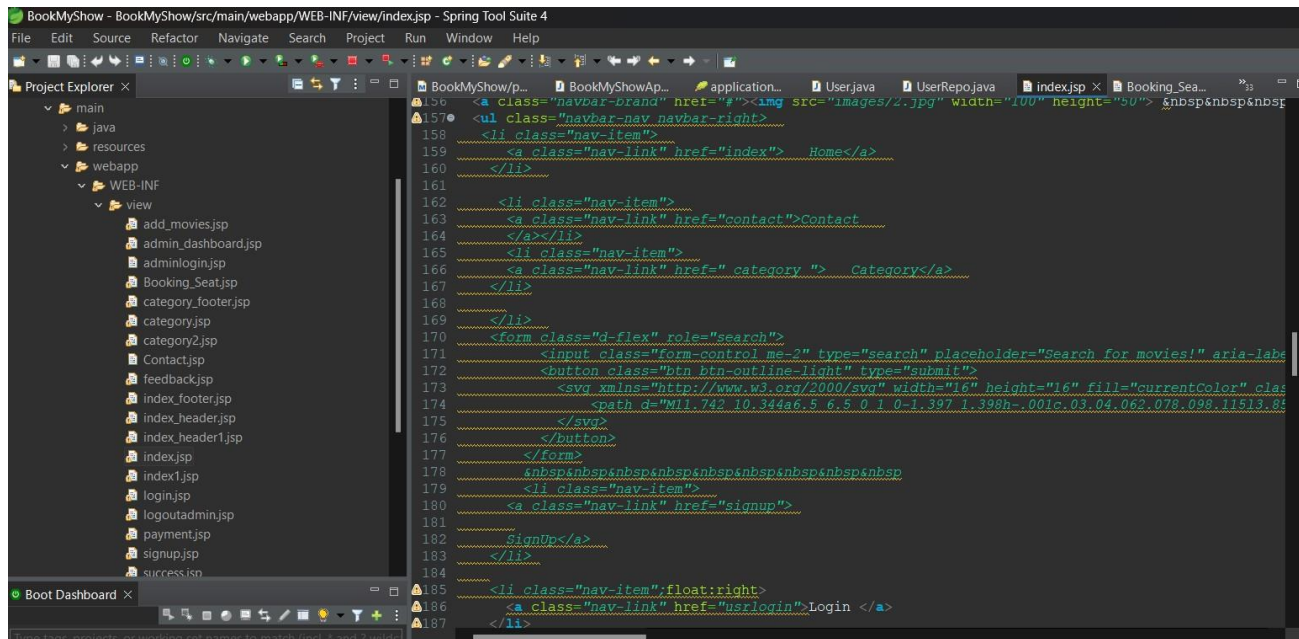
Logout Successful

You have been successfully logged out.

[Login again](#)

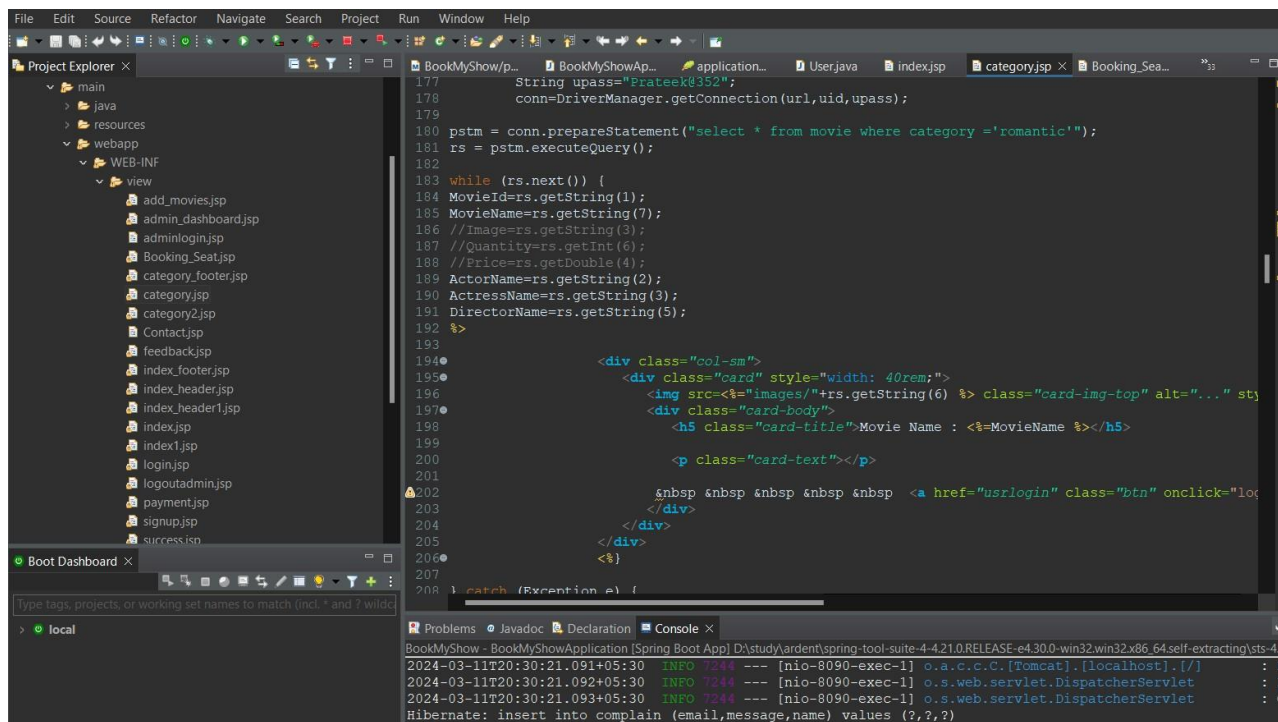
SOURCE CODE RELATED PROJECT :-

INDEX.JSP –



```
156 <a class="navbar-brand" href="#"></a></li>
157 <ul class="navbar-nav navbar-right">
158 <li class="nav-item">
159 <a class="nav-link" href="index"> Home</a>
160 </li>
161 <li class="nav-item">
162 <a class="nav-link" href="contact">Contact
163 </a></li>
164 <li class="nav-item">
165 <a class="nav-link" href="category"> Category</a>
166 </li>
167 </ul>
168 </div>
169 </div>
170 <form class="d-flex role="search">
171 <input class="form-control me-2" type="search" placeholder="Search for movies!" aria-label="Search">
172 <button class="btn btn-outline-light" type="submit">
173 <svg xmlns="http://www.w3.org/2000/svg" width="16" height="16" fill="currentColor" class="bi bi-search">
174 <path d="M11.742 10.344a6.5 6.5 0 1 0-1.397 1.398h-.001c.03.04.062.078.098.115l3.854 3.854c.375.375.938.375.938.001s-.375-.938-.375-.938z">
175 </svg>
176 </button>
177 </form>
178 <li class="nav-item">
179 <a class="nav-link" href="signup">
180 Sign Up</a>
181 </li>
182 <li class="nav-item">
183 <a class="nav-link" href="login">Login</a>
184 </li>
185 </ul>
186 </div>
187 </div>
```

CATEGORY.JSP



```
177 String upass="Prateek@352";
178 conn=DriverManager.getConnection(url,uid,upass);
179
180 pstmt = conn.prepareStatement("select * from movie where category ='romantic'");
181 rs = pstmt.executeQuery();
182
183 while (rs.next()) {
184 MovieId=rs.getString(1);
185 MovieName=rs.getString(7);
186 //Image=rs.getString(3);
187 //Quantity=rs.getInt(6);
188 //Price=rs.getDouble(4);
189 ActorName=rs.getString(2);
190 ActressName=rs.getString(3);
191 DirectorName=rs.getString(5);
192 %>
193
194 <div class="col-sm">
195 <div class="card" style="width: 40rem;">
196  class="card-img-top" alt="..." style="width: 100%; height: 100px; object-fit: cover;" />
197 <div class="card-body">
198 <h5 class="card-title">Movie Name : <%=MovieName %></h5>
199
200 <p class="card-text"></p>
201
202 <a href="usrlogin" class="btn" onclick="login()">Login</a>
203 </div>
204 </div>
205 </div>
206 <%>
207
208 } catch (Exception e) {
```

Problems | Javadoc | Declaration | Console

BookMyShow - BookMyShowApplication [Spring Boot App] D:\study\larent\spring-tool-suite-4-421.0.RELEASE-e430.0-win32-win32-x86_64-self-extracting-sts-4.1.0\sts-4.1.0\bin\sts-4.1.0.exe

2024-03-11T20:30:21.091+05:30 INFO 7244 --- [nio-8090-exec-1] o.a.c.c.C.[Tomcat].[localhost].[/] : ...

2024-03-11T20:30:21.092+05:30 INFO 7244 --- [nio-8090-exec-1] o.s.web.servlet.DispatcherServlet : ...

2024-03-11T20:30:21.093+05:30 INFO 7244 --- [nio-8090-exec-1] o.s.web.servlet.DispatcherServlet : ...

Hibernate: insert into complain (email,message,name) values (?, ?, ?)

SIGNUP.JSP –

```
Project: Save All (Ctrl+Shift+S)
  Booking_Seat.jsp
  category_footer.jsp
  category.jsp
  category2.jsp
  Contact.jsp
  feedback.jsp
  index_footer.jsp
  index_header.jsp
  index_header1.jsp
  index.jsp
  index1.jsp
  login.jsp
  logoutadmin.jsp
  payment.jsp
  signup.jsp
  success.jsp
  todofeature.jsp
  view_movies.jsp
  viewcontact.jsp
  viewfeedback.jsp
  viewuser.jsp
  > test
  > target
  HELP.md
  mvnw

Boot Dashboard X

28 <path d="M3 14s-1 0-1 1-4 6-4 6 3 6 4-1 1-1 1H3Zm5-6a3 3 0 1 0 0-6 3 3 0 0 0 0 6Z"/>
29 </svg>
30 <i class="fas fa-user fa-lg me-3 fa-fw"></i><input type="text" id="validationDefault01" name="uname" class="form-control" value="" re
31 <div class="form-outline flex-fill mb-0">
32 <label for="validationDefault01" class="form-label">Your Name</label>
33 <input type="text" id="validationDefault01" name="uname" class="form-control" value="" re
34
35 </div>
36 </div>
37
38
39 <div class="d-flex flex-row align-items-center mb-4">
40 <svg xmlns="http://www.w3.org/2000/svg" width="30" height="70" fill="currentColor" class="b
41 <path d="M.05 3.555A2 2 0 0 1 2 2h12a2 2 0 0 1 1.95 1.555L8 8.414.05 3.555ZM0 4.697v7.104
42 </svg>
43 <i class="fas fa-envelope fa-lg me-3 fa-fw"></i><input type="email" id="form3Example3c" name="uemail" class="form-control" />
44 <div class="form-outline flex-fill mb-0">
45 <label class="form-label" for="form3Example3c">Your Email</label>
46 <input type="email" id="form3Example3c" name="uemail" class="form-control" />
47
48 </div>
49 </div>
50
51 <div class="d-flex flex-row align-items-center mb-4">
52 <svg xmlns="http://www.w3.org/2000/svg" width="30" height="70" fill="currentColor" class="b
53 <path fill-rule="evenodd" d="M1.885 5.11a1.745 1.745 0 0 1 2.61 1.63L6.29 2.98c.329.423.445
54 </svg>
55
56 <i class="fas fa-envelope fa-lg me-3 fa-fw"></i><input type="text" id="form3Example3c" name="umhila" class="form-control" />
57 <div class="form-outline flex-fill mb-0">
58 <label class="form-label" for="form3Example3c">Your Mobile Number</label>
59 <input type="text" id="form3Example3c" name="umhila" class="form-control" />
```

ADMINLOGIN.JSP-

```
main
  > java
  > resources
  webapp
    WEB-INF
      view
        add_movies.jsp
        admin_dashboard.jsp
        adminlogin.jsp
        Booking_Seat.jsp
        category_footer.jsp
        category.jsp
        category2.jsp
        Contact.jsp
        feedback.jsp
        index_footer.jsp
        index_header.jsp
        index_header1.jsp
        index.jsp
        index1.jsp
        login.jsp
        logoutadmin.jsp
        payment.jsp
        signup.jsp
        success.jsp

Boot Dashboard X

22 text-align:center;
23 }
24 .sub{
25 background-color:blue;
26 padding: 7px 40px 7px 40px;
27 color:white;
28 font-weight:bold;
29 margin-left:70px;
30 border-radius:5px;
31 }
32 </style>
33 </head>
34 <body bgcolor="black" text="solid red">
35
36 <div class="loginform">
37 <h2 class="formheading">Admin Login Page</h2>
38 <form action="adminloginproc" method="POST">
39 <table>
40 <tr><td>Email:</td><td><input type="email" name="uemail"/></td></tr>
41 <tr><td>Password:</td><td><input type="password" name="upass"/></td></tr>
42 <tr><td colspan="2" style="text-align:center"><input class="sub" type="submit" value="login"/></td></tr>
43 </table>
44 <h3>{msg}</h3>
45 </form>
46
47 </div>
48
49 </body>
50 </html>
```

ADMINCONTROLLER.JAVA-

```
1 2
3  import org.springframework.beans.factory.annotation.Autowired;
4
5  11
6  12 @Controller
7  13 public class Admin_Controller {
8  14
9  15     @Autowired
10     16     Movie_Repo mrepo;
11     17
12     18     @RequestMapping("adminlogin")
13     19     public String adminlogin()
14     20     {
15     21         return "adminlogin";
16     22     }
17     23     @RequestMapping("adminloginproc")
18     24     public String adminLoginProc(@RequestParam("uemail")String email,@RequestParam("upass")String pas:
19     25     {
20     26         if(email.equals("admin@gmail.com") && pass.equals("admin"))
21     27         {
22     28             return "admin_dashboard";
23     29         }
24     30         else
25     31         {
26     32             model.put("msg", "email or password error!");
27     33             return "adminlogin";
28     34         }
29     35     }
30     36     @RequestMapping("admdash")
31     37     public String adminDashboard()
32     38     {
33     39         return "admin_dashboard";
34     40     }
35 }
```

USERCONTROLLER.JAVA-

```
1 2
3  import org.springframework.beans.factory.annotation.Autowired;
4
5  17
6  18 @Controller
7  19 public class Home_User_Controller {
8  20
9  21     @Autowired
10     22     UserRepo urepo;
11     23     @Autowired
12     24     ComplainRepo crepo;
13     25     @Autowired
14     26     Feedback Repo frepo;
15     27     @Autowired
16     28     BookingRepo brepo;
17     29     @RequestMapping("indexheader")
18     30     public String indexHeader()
19     31     {
20     32         return "index_header";
21     33     }
22     34     @RequestMapping("indexheader1")
23     35     public String indexHeader1()
24     36     {
25     37         return "index_header1";
26     38     }
27     39     @RequestMapping("indexfooter")
28     40     public String indexFooter()
29     41     {
30     42         return "index_footer";
31     43     }
32     44     @RequestMapping("index")
33     45     public String index()
34     46     {
35     47         return "index";
36     48     }
37 }
```

CONTACTADMIN.JAVA-

```
Project Explorer X
main
  java
    com
      movie
        controller
          Admin_Controller.java
          Home_User_Controller.java
        model
          Booking.java
          BookingRepo.java
          ComplainRepo.java
          ContactAdmin.java
          FeedbackRepo.java
          Feedback.java
          MovieRepo.java
          Movie.java
          User.java
          UserRepo.java
        service
          BookMyShowApplication.java
          ServletInitializer.java
      resources
      webapp
        WEB-INF
          view

Boot Dashboard X

1 package com.movie.model;
2 import jakarta.persistence.Column;
3
4
5
6
7
8
9 @Entity
10 @Table(name="Complain")
11 public class ContactAdmin {
12     @Id
13     @Column(name="id")
14     @GeneratedValue(strategy=GenerationType.IDENTITY)
15     private Integer id;
16
17     @Column(name="name")
18     private String name;
19     @Column(name="email")
20     private String email;
21
22     @Column(name="message")
23     private String message;
24
25     public Integer getId() {
26         return id;
27     }
28
29     public void setId(Integer id) {
30         this.id = id;
31     }
32
33     public String getName() {
34         return name;
35     }
36
37     public void setName(String name) {
38         this.name = name;
39     }
40
41     public String getEmail() {
42         return email;
43     }
44
45     public void setEmail(String email) {
46         this.email = email;
47     }
48
49     public String getMessage() {
50         return message;
51     }
52
53     public void setMessage(String message) {
54         this.message = message;
55     }
56
57 }
```

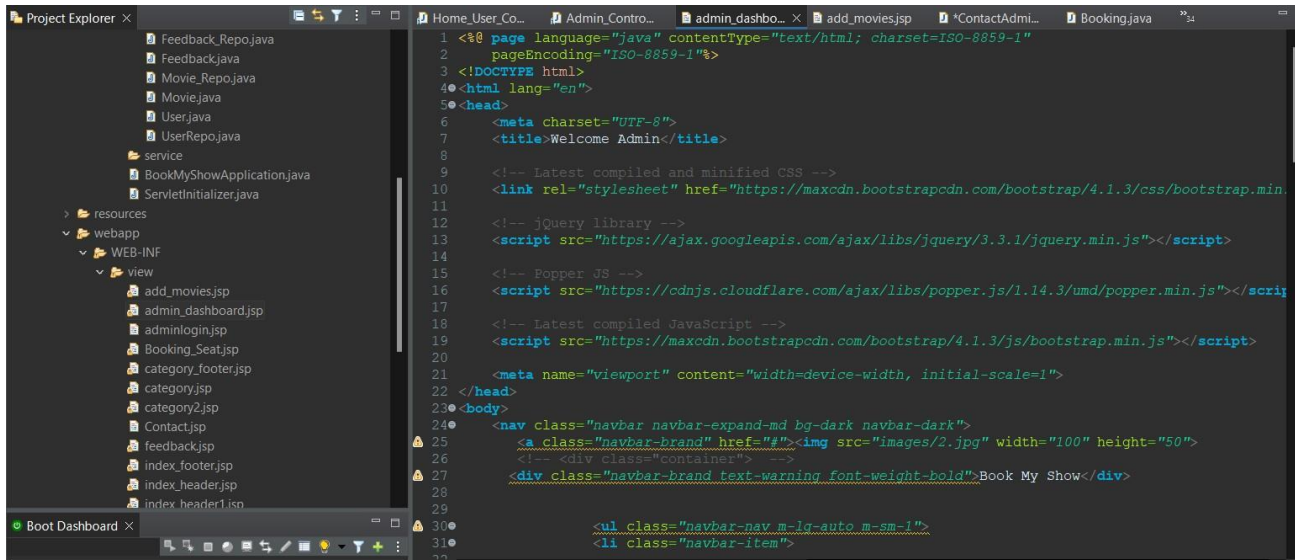
ADDMOVIE.JSP-

```
Project Explorer X
FeedbackRepo.java
Feedback.java
MovieRepo.java
Movie.java
User.java
UserRepo.java
service
BookMyShowApplication.java
ServletInitializer.java
resources
webapp
  WEB-INF
    view
      add_movies.jsp
      admin_dashboard.jsp
      adminlogin.jsp
      Booking_Seat.jsp
      category_footer.jsp
      category.jsp
      category2.jsp
      Contact.jsp
      feedback.jsp
      index_footer.jsp
      index_header.jsp
      index_header1.jsp

Boot Dashboard X

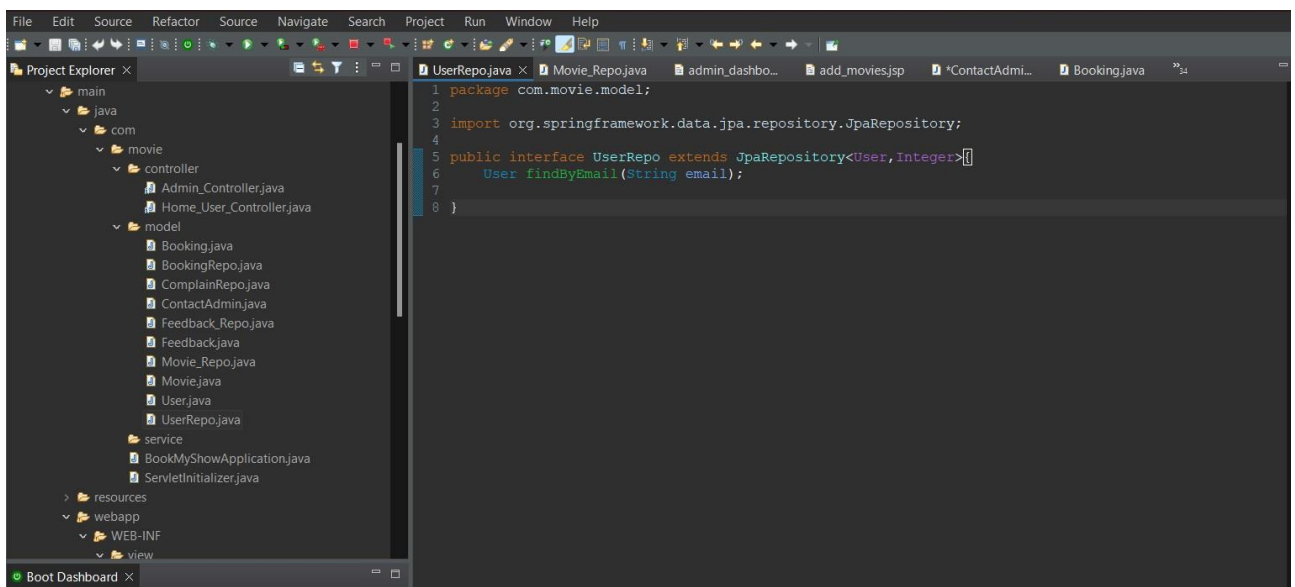
47 }
48 </style>
49 <body>
50
51 <h3>Add Movies</h3>
52
53 <div>
54 <form action="movi" method="post" "multipart/form-data" >
55 <label for="fname">Movie Id</label>
56 <input type="text" name="movieid" placeholder="Movie Id..">
57
58 <label for="lname">Movie Name</label>
59 <input type="text" name="moviename" placeholder="Movie name..">
60
61 <label for="country">Category</label>
62 <select id="country" name="category">
63 <option value="romantic">Romantic</option>
64 <option value="horror">Horror</option>
65 <option value="thriller">Thriller</option>
66 <option value="scifi">Sci-Fi</option>
67 <option value="comedy">Comedy</option>
68 <option value="Action">Action</option>
69 </select>
70 <label for="lname">Ticket Quantity</label>
71 <input type="text" name="qty" placeholder="Quantity..">
72
73 <label for="lname">Price</label>
74 <input type="text" name="price" placeholder="Price..">
75 <label for="lname">Actors</label>
76 <input type="text" name="actor" placeholder="Actor..">
77 <label for="lname">Actress</label>
78 <input type="text" name="actress" placeholder="Actress..">
79 </div>
80 </body>
81 </html>
```


ADMINDASHBOARD.JSP –



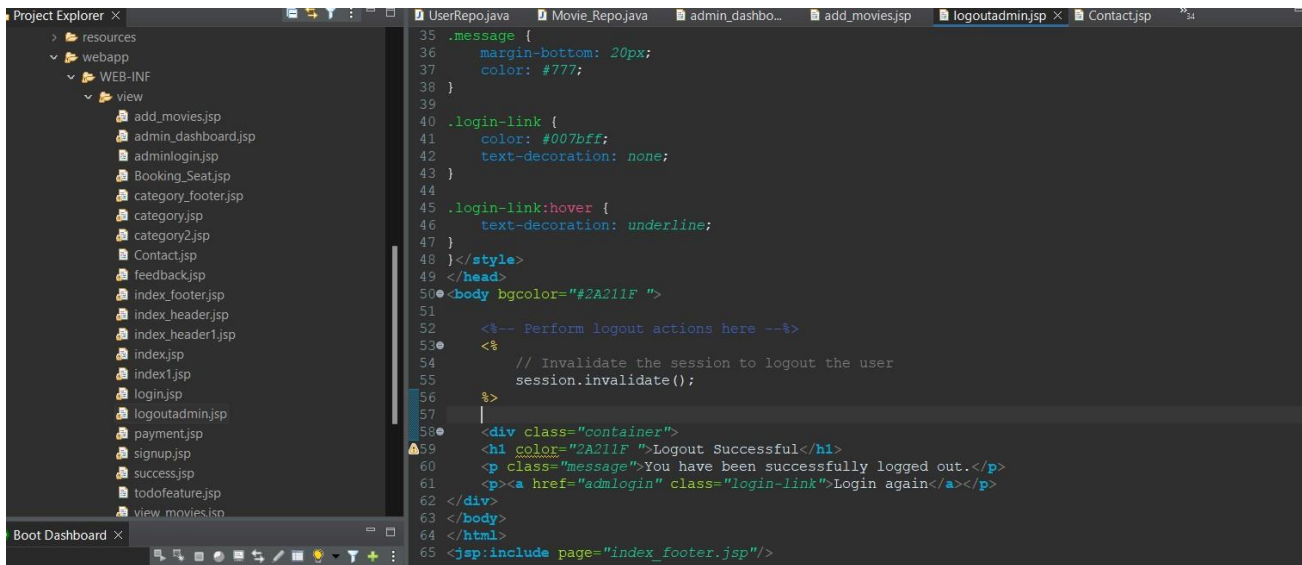
```
1 <%@ page language="java" contentType="text/html; charset=ISO-8859-1"
2     pageEncoding="ISO-8859-1"%>
3 <!DOCTYPE html>
4 <html lang="en">
5 <head>
6     <meta charset="UTF-8">
7     <title>Welcome Admin</title>
8
9     <!-- Latest compiled and minified CSS -->
10    <link rel="stylesheet" href="https://maxcdn.bootstrapcdn.com/bootstrap/4.1.3/css/bootstrap.min.
11
12    <!-- jQuery library -->
13    <script src="https://ajax.googleapis.com/ajax/libs/jquery/3.3.1/jquery.min.js"></script>
14
15    <!-- Popper JS -->
16    <script src="https://cdnjs.cloudflare.com/ajax/libs/popper.js/1.14.3/umd/popper.min.js"></scrip
17
18    <!-- Latest compiled JavaScript -->
19    <script src="https://maxcdn.bootstrapcdn.com/bootstrap/4.1.3/js/bootstrap.min.js"></script>
20
21    <meta name="viewport" content="width=device-width, initial-scale=1">
22 </head>
23 <body>
24     <nav class="navbar navbar-expand-md bg-dark navbar-dark">
25         <a class="navbar-brand" href="#">
26         <!-- <div class="container"> -->
27         <div class="navbar-brand text-warning font-weight-bold">Book My Show</div>
28
29     <ul class="navbar-nav m-lq-auto m-sm-1">
30     <li class="navbar-item">
```

USERREPO(INTERFACE) –



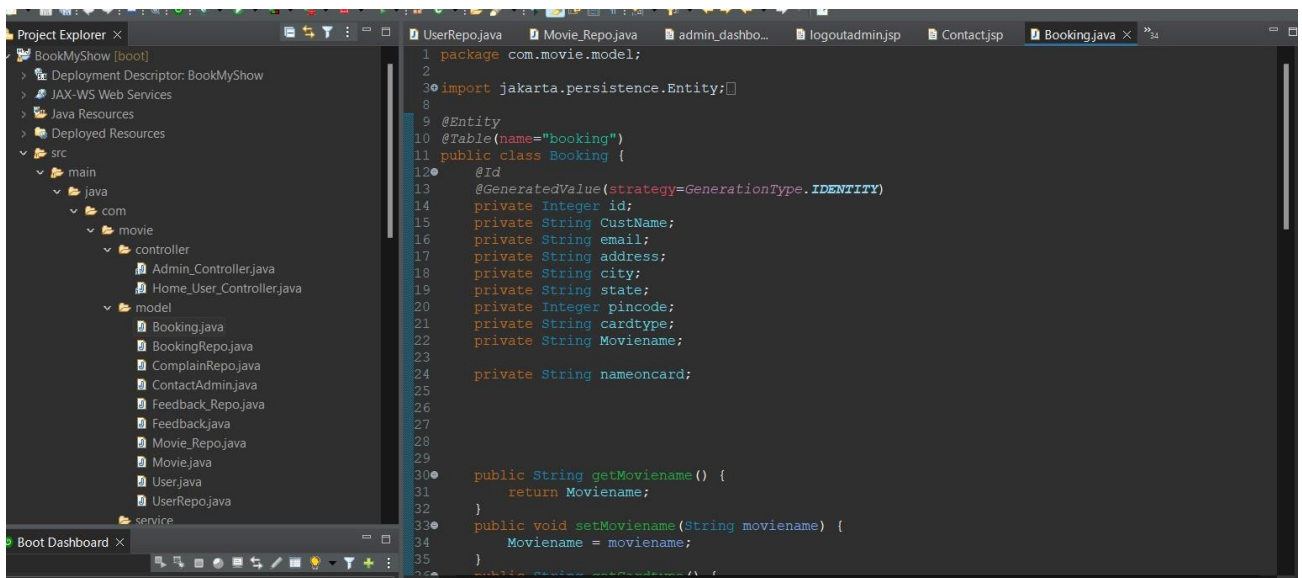
```
1 package com.movie.model;
2
3 import org.springframework.data.jpa.repository.JpaRepository;
4
5 public interface UserRepo extends JpaRepository<User,Integer>{
6     User findByEmail(String email);
7
8 }
```

LOGOUTADMIN.JSP –



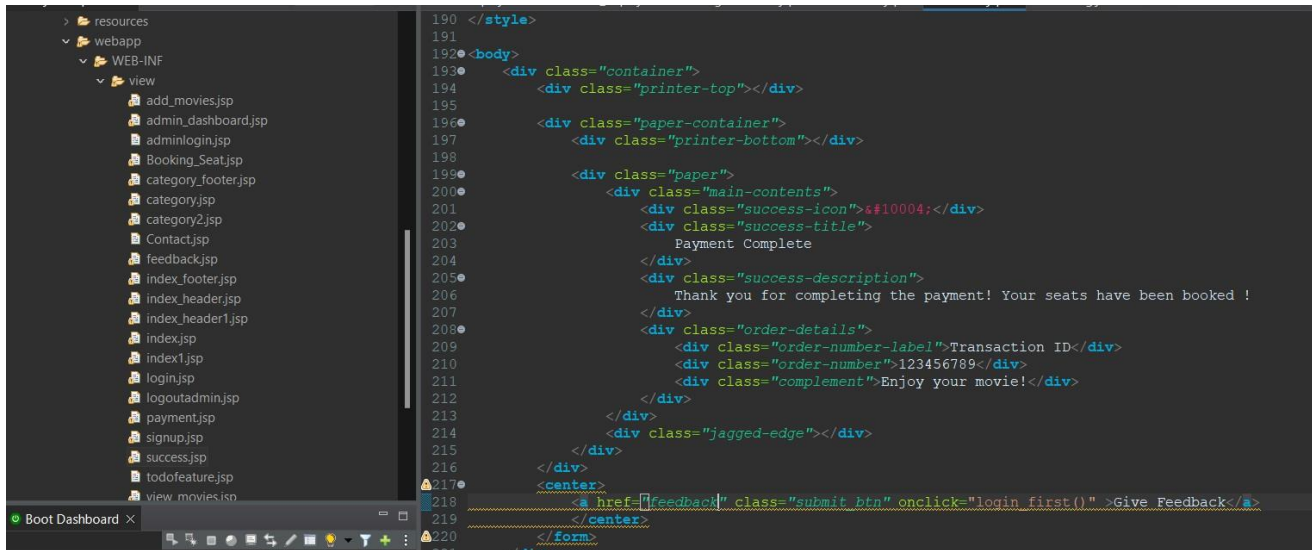
```
35 .message {
36     margin-bottom: 20px;
37     color: #777;
38 }
39
40 .login-link {
41     color: #007bff;
42     text-decoration: none;
43 }
44
45 .login-link:hover {
46     text-decoration: underline;
47 }
48 }</style>
49 </head>
50 <body bgcolor="#2A211F ">
51
52     <!-- Perform logout actions here -->
53     <%
54         // Invalidate the session to logout the user
55         session.invalidate();
56     %>
57
58     <div class="container">
59         <h1 color="#2A211F ">Logout Successful</h1>
60         <p class="message">You have been successfully logged out.</p>
61         <p><a href="adminlogin" class="login-link">Login again</a></p>
62     </div>
63 </body>
64 </html>
65 <jsp:include page="index_footer.jsp"/>
```

BOOKING.JAVA –



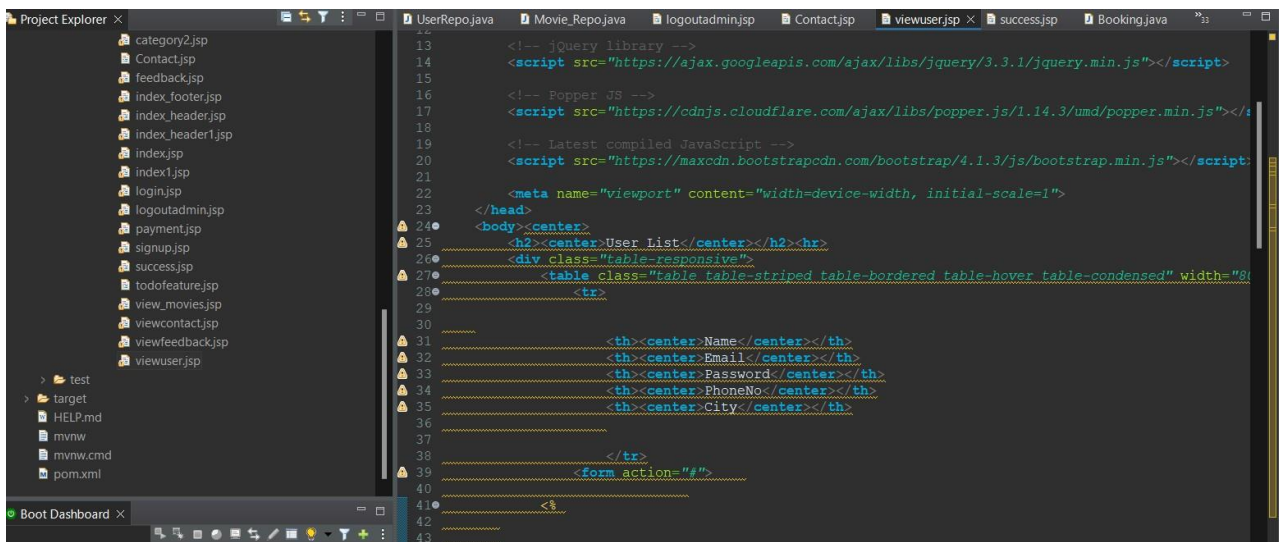
```
1 package com.movie.model;
2
3 import jakarta.persistence.Entity;
4
5
6
7
8
9 @Entity
10 @Table(name="booking")
11 public class Booking {
12     @Id
13     @GeneratedValue(strategy=GenerationType.IDENTITY)
14     private Integer id;
15     private String CustName;
16     private String email;
17     private String address;
18     private String city;
19     private String state;
20     private Integer pincode;
21     private String cardtype;
22     private String Moviename;
23
24     private String nameoncard;
25
26
27
28
29
30     public String getMoviename() {
31         return Moviename;
32     }
33     public void setMoviename(String moviename) {
34         Moviename = moviename;
35     }
36 }
```

SUCCESS.JSP –



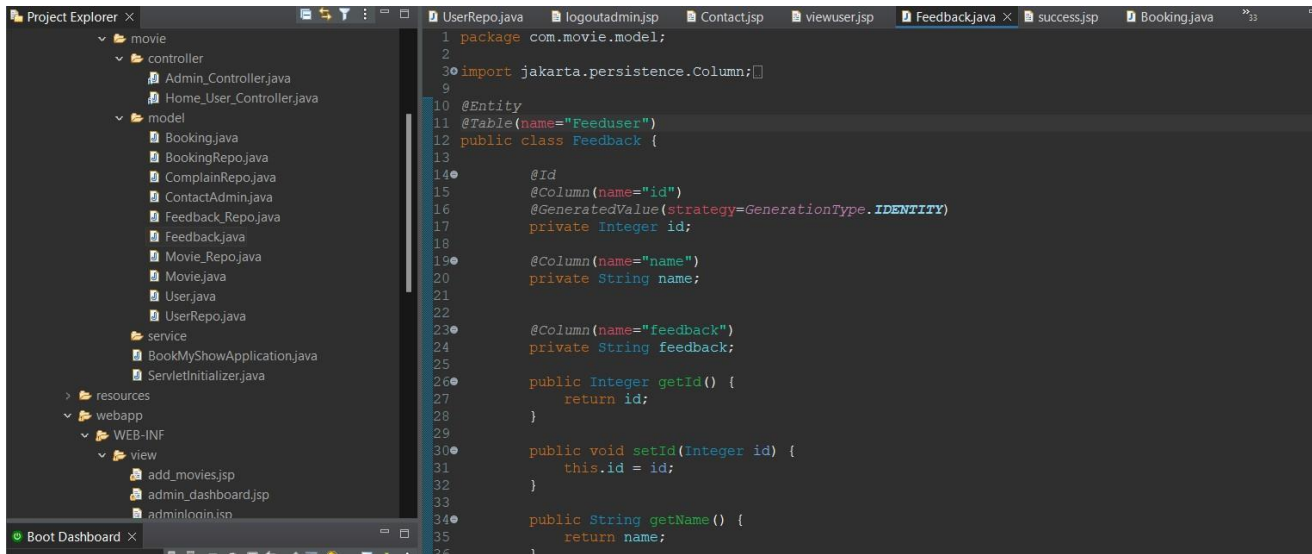
```
190 </style>
191
192 <body>
193   <div class="container">
194     <div class="printer-top"></div>
195
196     <div class="paper-container">
197       <div class="printer-bottom"></div>
198
199       <div class="paper">
200         <div class="main-contents">
201           <div class="success-icon">✔</div>
202           <div class="success-title">
203             Payment Complete
204           </div>
205           <div class="success-description">
206             Thank you for completing the payment! Your seats have been booked !
207           </div>
208           <div class="order-details">
209             <div class="order-number-label">Transaction ID</div>
210             <div class="order-number">123456789</div>
211             <div class="complement">Enjoy your movie!</div>
212           </div>
213         </div>
214         <div class="jagged-edge"></div>
215       </div>
216     </div>
217   </body>
218   <center>
219     <a href="#feedback" class="submit btn" onclick="login_first()" >Give Feedback</a>
220   </center>
221 </form>
222
```

VIEWUSER.JSP –



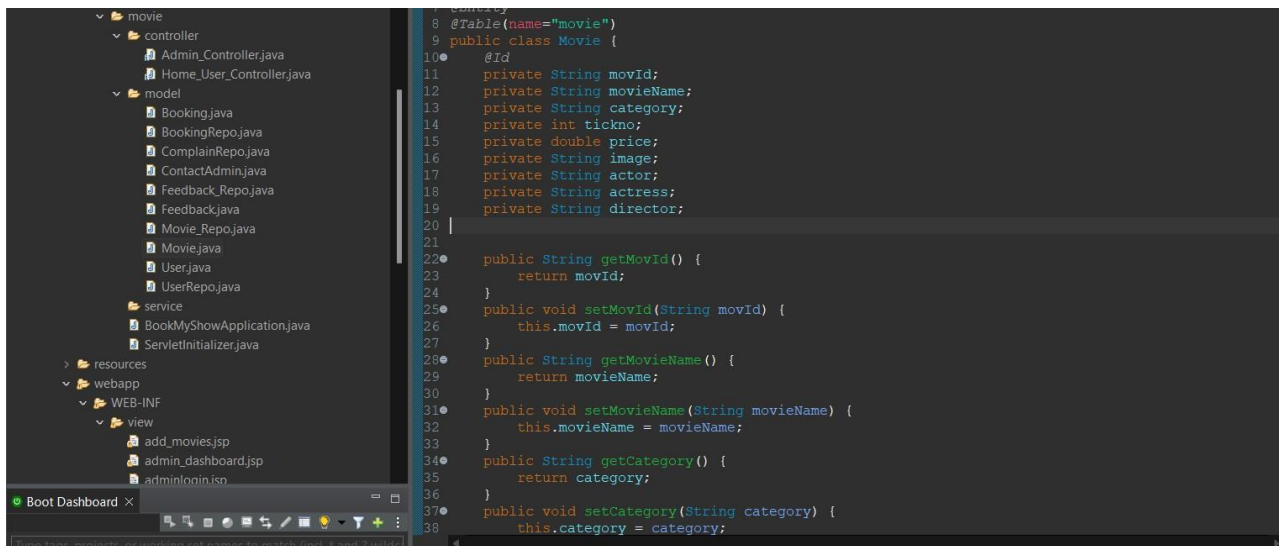
```
13 <!-- jQuery library -->
14 <script src="https://ajax.googleapis.com/ajax/libs/jquery/3.3.1/jquery.min.js"></script>
15
16 <!-- Popper JS -->
17 <script src="https://cdnjs.cloudflare.com/ajax/libs/popper.js/1.14.3/umd/popper.min.js"></script>
18
19 <!-- Latest compiled JavaScript -->
20 <script src="https://maxcdn.bootstrapcdn.com/bootstrap/4.1.3/js/bootstrap.min.js"></script>
21
22 <meta name="viewport" content="width=device-width, initial-scale=1">
23
24 </head>
25 <body><center>
26   <h2>User List</h2></center></div>
27   <div class="table-responsive">
28     <table class="table table-striped table-bordered table-hover table-condensed" width="80">
29       <tr>
30
31         <th><center>Name</center></th>
32         <th><center>Email</center></th>
33         <th><center>Password</center></th>
34         <th><center>PhoneNo</center></th>
35         <th><center>City</center></th>
36       </tr>
37
38       <tr>
39         <td><input type="text" value="">
40       </td>
41     </tr>
42   </div>
43
```

FEEDBACK.JAVA –



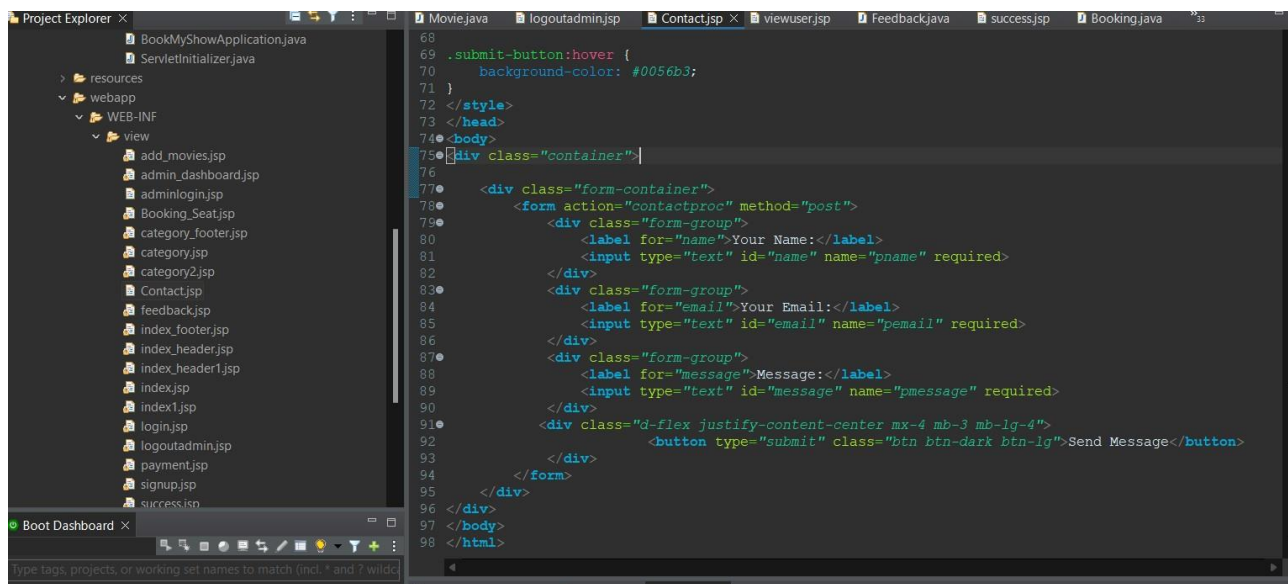
```
1 package com.movie.model;
2
3 import jakarta.persistence.Column;
4
5
6
7
8
9
10 @Entity
11 @Table(name="Feeduser")
12 public class Feedback {
13
14     @Id
15     @Column(name="id")
16     @GeneratedValue(strategy=GenerationType.IDENTITY)
17     private Integer id;
18
19     @Column(name="name")
20     private String name;
21
22
23     @Column(name="feedback")
24     private String feedback;
25
26     public Integer getId() {
27         return id;
28     }
29
30     public void setId(Integer id) {
31         this.id = id;
32     }
33
34     public String getName() {
35         return name;
36     }
37
38 }
```

MOVIE.JAVA –



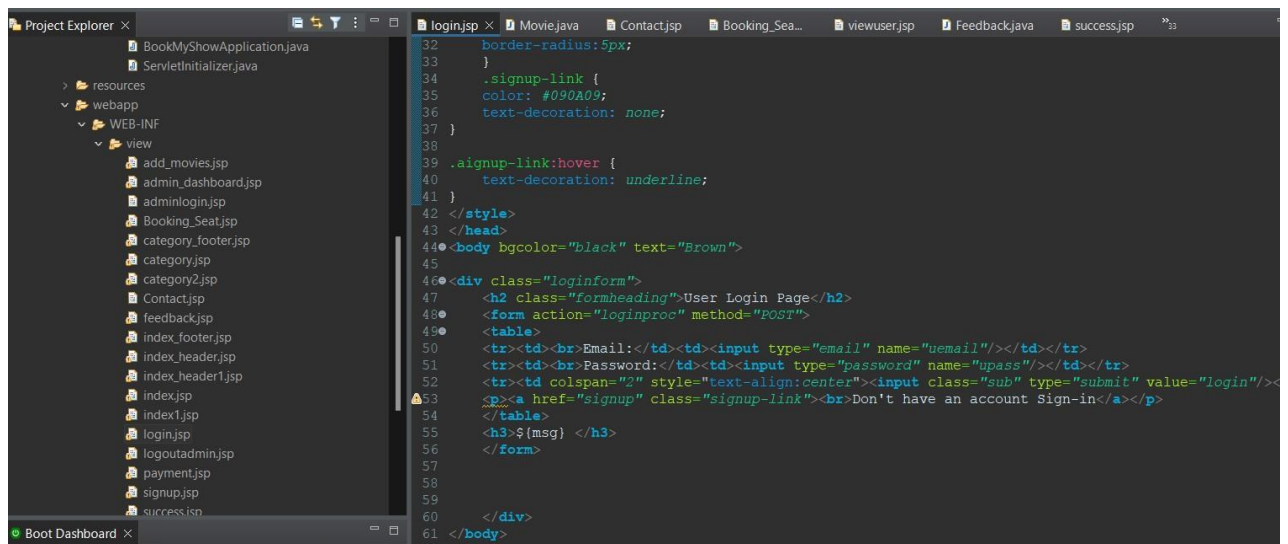
```
1 package com.movie.model;
2
3 import jakarta.persistence.Column;
4
5
6
7
8 @Entity
9 @Table(name="movie")
10 public class Movie {
11
12     @Id
13     private String movId;
14     private String movieName;
15     private String category;
16     private int tickno;
17     private double price;
18     private String image;
19     private String actor;
20     private String actress;
21     private String director;
22
23     public String getMovId() {
24         return movId;
25     }
26     public void setMovId(String movId) {
27         this.movId = movId;
28     }
29     public String getMovieName() {
30         return movieName;
31     }
32     public void setMovieName(String movieName) {
33         this.movieName = movieName;
34     }
35     public String getCategory() {
36         return category;
37     }
38     public void setCategory(String category) {
39         this.category = category;
40     }
41
42 }
```


CONTACT.JSP –



```
68
69 .submit-button:hover {
70     background-color: #0056b3;
71 }
72 </style>
73 </head>
74 <body>
75 <div class="container">
76
77 <div class="form-container">
78 <form action="contactproc" method="post">
79 <div class="form-group">
80 <label for="name">Your Name:</label>
81 <input type="text" id="name" name="pname" required>
82 </div>
83 <div class="form-group">
84 <label for="email">Your Email:</label>
85 <input type="text" id="email" name="pemail" required>
86 </div>
87 <div class="form-group">
88 <label for="message">Message:</label>
89 <input type="text" id="message" name="pmessage" required>
90 </div>
91 <div class="d-flex justify-content-center mx-4 mb-3 mb-lg-4">
92 <button type="submit" class="btn btn-dark btn-lg">Send Message</button>
93 </div>
94 </form>
95 </div>
96 </div>
97 </body>
98 </html>
```

USER (LOGIN.JP) –



```
32 border-radius:5px;
33 }
34 .signup-link {
35 color: #090A09;
36 text-decoration: none;
37 }
38
39 .signup-link:hover {
40 text-decoration: underline;
41 }
42 </style>
43 </head>
44 <body bgcolor="black" text="Brown">
45
46 <div class="loginform">
47 <h2 class="formheading">User Login Page</h2>
48 <form action="loginproc" method="POST">
49 <table>
50 <tr><td><br>Email:</td><td><input type="email" name="uemail"/></td></tr>
51 <tr><td><br>Password:</td><td><input type="password" name="upass"/></td></tr>
52 <tr><td colspan="2" style="text-align:center"><input class="sub" type="submit" value="login"/>
53 </td></tr>
54 </table>
55 <p><a href="signup" class="signup-link"><br>Don't have an account Sign-in</a></p>
56 </form>
57
58
59
60 </div>
61 </body>
```

BOOKING SEAT.JSP -

```
200 <td>5</td>
201 <td></td>
202 <td>6</td>
203 <td>7</td>
204 <td>8</td>
205 <td>9</td>
206 <td>10</td>
207 <td>11</td>
208 <td>12</td>
209 </tr>
210
211 <tr>
212 <td>A</td>
213 <td><input type="checkbox" class="seats" value="A1"></td>
214 <td><input type="checkbox" class="seats" value="A2"></td>
215 <td><input type="checkbox" class="seats" value="A3"></td>
216 <td><input type="checkbox" class="seats" value="A4"></td>
217 <td><input type="checkbox" class="seats" value="A5"></td>
218 <td class="seatGap"></td>
219 <td><input type="checkbox" class="seats" value="A6"></td>
220 <td><input type="checkbox" class="seats" value="A7"></td>
221 <td><input type="checkbox" class="seats" value="A8"></td>
222 <td><input type="checkbox" class="seats" value="A9"></td>
223 <td><input type="checkbox" class="seats" value="A10"></td>
224 <td><input type="checkbox" class="seats" value="A11"></td>
225 <td><input type="checkbox" class="seats" value="A12"></td>
226 </tr>
227
228 <tr>
229 <td>B</td>
230 <td><input type="checkbox" class="seats" value="B1"></td>
```

THANK YOU